

Snowflake Projects Vault — PJ's Academy

20 hands-on projects from beginner pipelines to real-time streaming platforms and ML-in-warehouse systems. Every project maps to real job responsibilities and SnowPro exam domains.

Difficulty & Domain Map

#	Project	Level	Domain
01	First Data Warehouse	★	Fundamentals
02	Multi-Source Data Loader	★★	Loading
03	Semi-Structured JSON Analytics	★★	VARIANT
04	Performance Tuning Lab	★★★	Optimisation
05	Time Travel Recovery System	★★	Time Travel
06	Zero-Copy Dev/Test Factory	★★★	Cloning
07	Real-Time Streaming Pipeline	★★★★	Streams/Tasks
08	Snowpipe Auto-Ingestion	★★★	Snowpipe
09	Snowpark ML Pipeline	★★★★	Snowpark
10	In-Warehouse ML Model	★★★★	Snowpark ML
11	RBAC Security Framework	★★★	Governance
12	Data Masking & Compliance	★★★	Security
13	Cost Optimisation Dashboard	★★★	FinOps
14	dbt + Snowflake Transformation	★★★★	Modern Stack
15	CDC Data Vault	★★★★	Data Modelling
16	Data Sharing Marketplace	★★★	Sharing
17	Real-Time Dashboard (Streamlit)	★★★★	Apps
18	Fraud Detection in Snowflake	★★★★★	ML + Streaming
19	Multi-Region DR Platform	★★★★★	Architecture
20	End-to-End Data Platform	★★★★★	Capstone

01 — First Data Warehouse ★

Build: A complete sales data warehouse — dimensional model, warehouse, loading, and analytics queries.

```
CREATE WAREHOUSE analytics_wh WAREHOUSE_SIZE='XSMALL' AUTO_SUSPEND=60 AUTO_RESUME=TRUE;
CREATE DATABASE retail_dw;
CREATE SCHEMA retail_dw.star;

-- Star schema: fact + dimensions
CREATE TABLE dim_customer (customer_id INT, name STRING, city STRING, segment STRING);
CREATE TABLE dim_product (product_id INT, name STRING, category STRING, price NUMBER);
CREATE TABLE dim_date (date_id DATE, year INT, quarter INT, month INT, weekday STRING);
CREATE TABLE fact_sales (sale_id INT, customer_id INT, product_id INT,
                          date_id DATE, quantity INT, revenue NUMBER);
```

Outcome: You understand dimensional modelling and Snowflake basics.

02 — Multi-Source Data Loader ★★

Build: Load from CSV (S3), JSON (API dumps), and Parquet into one unified schema with error handling and validation. - File formats for each source type - `COPY INTO` with `ON_ERROR`, `VALIDATION_MODE` - A rejection table for bad records - **Innovation:** Auto-reconciliation report showing loaded vs rejected counts.

03 — Semi-Structured JSON Analytics ★★

Build: Analyse nested JSON (e-commerce events, IoT, API logs) without flattening it first — using `VARIANT`, `FLATTEN`, and path notation.

```
SELECT
  event:user_id::STRING AS user_id,
  event:type::STRING AS event_type,
  item.value:product::STRING AS product,
  item.value:price::NUMBER AS price
FROM raw_events,
LATERAL FLATTEN(input => event:cart_items) item
WHERE event:type::STRING = 'purchase';
```

Innovation: Query nested JSON as fast as structured tables — no ETL.

04 — Performance Tuning Lab ★★★

Build: Take slow queries and make them 10x faster. Clustering keys, search optimisation, materialised views, and result caching. - Read query profiles to find bottlenecks - Add clustering keys and measure pruning improvement - Compare materialised vs regular views - **Innovation:** A before/after benchmark report proving each optimisation's impact.

05 — Time Travel Recovery System ★★

Build: A "data undo" system — recover from accidental deletes, updates, and drops using Time Travel.

```
-- Someone ran a bad UPDATE. Recover the old values:
CREATE TABLE orders_recovered AS
SELECT * FROM orders BEFORE(STATEMENT => '<bad_query_id>');
```

```
UNDROP TABLE accidentally_dropped_table;
```

Innovation: An automated "recovery runbook" script for common disasters.

06 — Zero-Copy Dev/Test Factory ★★☆☆

Build: Instant dev/test/staging environments cloned from production — no storage cost, ready in seconds.

```
CREATE DATABASE dev_env CLONE production;
CREATE DATABASE test_env CLONE production;
-- Developers get full prod data instantly; only changes consume storage
```

Innovation: A self-service cloning script that spins up per-developer sandboxes.

07 — Real-Time Streaming Pipeline ★★★★★

Build: A near-real-time ELT pipeline using Streams + Tasks — data flows from raw → cleaned → aggregated automatically every minute.

```
CREATE STREAM raw_stream ON TABLE raw_orders;

CREATE TASK clean_task WAREHOUSE=etl_wh SCHEDULE='1 MINUTE'
WHEN SYSTEM$STREAM_HAS_DATA('raw_stream')
AS
  INSERT INTO clean_orders
  SELECT order_id, TRIM(customer), ABS(amount), order_ts
  FROM raw_stream WHERE amount IS NOT NULL;

-- Chained task: aggregate after cleaning
CREATE TASK agg_task WAREHOUSE=etl_wh AFTER clean_task
AS
  MERGE INTO daily_summary d USING (
    SELECT DATE(order_ts) dt, SUM(amount) total FROM clean_orders GROUP BY 1
  ) s ON d.dt = s.dt
  WHEN MATCHED THEN UPDATE SET total = s.total
  WHEN NOT MATCHED THEN INSERT VALUES (s.dt, s.total);
```

Innovation: A DAG of chained tasks — a full pipeline with no external orchestrator.

08 — Snowpipe Auto-Ingestion ★★☆☆

Build: Files dropped in S3 auto-load within seconds via Snowpipe + event notifications. - Configure S3 event notifications → SQS → Snowpipe - Monitor pipe status and load history - **Innovation:** A monitoring query that alerts on ingestion lag.

09 — Snowpark ML Pipeline ★★★★★

Build: Feature engineering + model training entirely in Snowpark Python — data never leaves Snowflake.

```

from snowflake.snowpark import Session
from snowflake.snowpark.functions import col, datediff, current_date

df = session.table("customers")
features = (df
    .with_column("tenure_days", datediff("day", col("signup_date"), current_date()))
    .with_column("high_value", (col("total_spend") > 10000).cast("int"))
    .select("customer_id", "tenure_days", "high_value", "total_spend"))

features.write.mode("overwrite").save_as_table("customer_features")

```

Innovation: Zero data egress — feature pipelines run on Snowflake compute.

10 — In-Warehouse ML Model ★★☆☆

Build: Train a scikit-learn/XGBoost model and deploy it as a Snowflake UDF so predictions run inside SQL.

```

@udf(name="predict_churn", is_permanent=True, stage_location="@ml_stage",
    packages=["scikit-learn", "pandas"])
def predict_churn(tenure: int, monthly: float, contract: int) -> float:
    import pickle
    model = pickle.load(open("model.pkl", "rb"))
    return float(model.predict_proba([[tenure, monthly, contract]])[0][1])

```

```

-- Now predict directly in SQL!
SELECT customer_id, predict_churn(tenure, monthly_charges, contract) AS churn_risk
FROM customers WHERE churn_risk > 0.7;

```

Innovation: ML inference as a SQL function — analysts use ML without Python.

11 — RBAC Security Framework ★★☆☆

Build: A complete role hierarchy for an organisation — functional roles, access roles, and least-privilege design. - Functional roles (analyst, engineer, admin) + access roles (read/write per schema) - Role inheritance hierarchy - **Innovation:** An audit query that maps every user to their effective permissions.

12 — Data Masking & Compliance ★★☆☆

Build: GDPR/DPDP-compliant PII protection — dynamic masking, row-level security, and access auditing.

```

CREATE MASKING POLICY pan_mask AS (val STRING) RETURNS STRING ->
CASE WHEN CURRENT_ROLE() IN ('COMPLIANCE') THEN val
ELSE 'XXXX' || RIGHT(val, 4) END;

CREATE MASKING POLICY aadhaar_mask AS (val STRING) RETURNS STRING ->
CASE WHEN CURRENT_ROLE() IN ('COMPLIANCE') THEN val
ELSE 'XXXX-XXXX-' || RIGHT(val, 4) END;

```

Innovation: India-specific masking (PAN, Aadhaar) for DPDP Act compliance.

13 — Cost Optimisation Dashboard ★★ ★

Build: A FinOps dashboard tracking credit usage, expensive queries, idle warehouses, and storage costs.

```
-- Warehouses burning credits while barely used
SELECT warehouse_name,
       SUM(credits_used) AS credits,
       COUNT(*) AS query_count,
       SUM(credits_used)/NULLIF(COUNT(*),0) AS credits_per_query
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY m
JOIN SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY q USING (warehouse_name)
GROUP BY 1 ORDER BY credits DESC;
```

Innovation: An automated "cost anomaly" detector flagging unusual spend spikes.

14 — dbt + Snowflake Transformation ★★ ★★

Build: A production ELT project with dbt — staging → intermediate → marts, with tests, docs, and incremental models. - Layered model architecture (staging/marts) - Incremental models for large tables - Data tests (unique, not_null, relationships) + auto-generated docs - **Innovation:** Slowly-changing-dimension (SCD Type 2) via dbt snapshots.

15 — CDC Data Vault ★★ ★★

Build: A Data Vault 2.0 model (hubs, links, satellites) with change-data-capture using Streams — the enterprise gold standard. - Hubs (business keys), Links (relationships), Satellites (attributes + history) - Stream-driven incremental loads - **Innovation:** Full historical audit trail — every change ever, queryable.

16 — Data Sharing Marketplace ★★ ★

Build: Share live data with partners (no copying) and consume Snowflake Marketplace datasets to enrich your own.

```
CREATE SHARE partner_share;
GRANT USAGE ON DATABASE analytics TO SHARE partner_share;
GRANT SELECT ON TABLE analytics.public.sales_summary TO SHARE partner_share;
ALTER SHARE partner_share ADD ACCOUNTS = partner_account;
```

Innovation: Enrich internal data with free Marketplace weather/demographic data via live joins.

17 — Real-Time Dashboard (Streamlit in Snowflake) ★★ ★★

Build: A live analytics dashboard running natively inside Snowflake with Streamlit — no external hosting.

```
import streamlit as st
from snowflake.snowpark.context import get_active_session

session = get_active_session()
st.title("❄️ Live Sales Dashboard — PJ's Academy")

df = session.sql("SELECT region, SUM(revenue) rev FROM sales GROUP BY region").to_pandas()
```

```
st.bar_chart(df.set_index("REGION"))

metric = session.sql("SELECT SUM(revenue) FROM sales").collect()[0][0]
st.metric("Total Revenue", f"${metric:,.0f}")
```

Innovation: Dashboard + data + compute all in one platform — zero data movement.

18 — Fraud Detection in Snowflake ★ ★ ★ ★ ★

Build: Real-time fraud scoring — transactions stream in, a Snowpark ML model scores them, and Tasks flag suspicious ones automatically. - Streaming ingestion (Snowpipe) → feature computation (Streams/Tasks) - In-warehouse ML UDF scoring - Alert table + notification on high-risk transactions - **Innovation:** Sub-minute fraud detection entirely within Snowflake — no external ML infra.

19 — Multi-Region DR Platform ★ ★ ★ ★ ★

Build: A disaster-recovery architecture with database replication across regions and automated failover. - Cross-region database replication - Failover groups - Replication lag monitoring - **Innovation:** A tested failover runbook with RPO/RTO measurement.

20 — End-to-End Data Platform (Capstone) ★ ★ ★ ★ ★

Build: The full stack — ingestion (Snowpipe) → transformation (dbt) → CDC (Streams/Tasks) → ML (Snowpark) → serving (Streamlit) → governance (RBAC/masking) → cost control (monitors).

This single project demonstrates **every SnowPro Advanced domain**. Ship it, document it, and it becomes the centrepiece of your data-engineering portfolio.

Architecture:

```
S3 → Snowpipe → RAW → Streams+Tasks → STAGING → dbt → MARTS
                        ↓
                Snowpark ML → Predictions → Streamlit Dashboard
                        ↓
RBAC + Masking (governance) | Resource Monitors (cost)
```

Innovation: A production-grade platform one person can build and operate — proving true full-stack data-engineering capability.

🎯 How to Use This Vault

1. **Beginners:** Projects 01–06 build your foundation.
2. **Job-ready:** Projects 07–14 cover what data engineering roles actually need.
3. **Senior-level:** Projects 15–20 prove architecture and ML skills.

Complete **projects 01, 07, 10, 14, and 20** and you have a portfolio that clears most Snowflake interviews.

Course: ❄️ *Snowflake Mastery* — *PJ's Academy*